=== UNIT 3: DATA STRUCTURES: STRINGS, LISTS, TUPLES & DICTS ===

>> 3.1 Strings: Architecture, Slicing and Methods

Strings in Python are immutable sequences of Unicode characters (str):
* Indexing and Extended Slicing: s[start:stop:step]. Positive indices index from 0 to N-1; negative indices index from -1 (last character) to -N. Step specifies stride: s[::-1] reverses a string in O(n) time.
* Core String Methods: .strip(), .lower(), .upper(), .split(sep), .join(iterable), .replace(old, new), .find(sub), .count(sub), .startswith(prefix), .endswith(suffix).

>> 3.2 Lists: Dynamic Array Architecture and Operations

Lists are mutable, ordered sequences of heterogeneous elements implemented internally as dynamic arrays of pointers:
* Memory Allocation: CPython overallocates list capacity geometrically (growth factor ~1.125) to achieve amortized O(1) append performance.
* Core List Methods: .append(x) [O(1)], .extend(iterable) [O(k)], .insert(i, x) [O(n)], .pop([i]) [O(1) from end, O(n) from start], .remove(x) [O(n)], .sort(key=None, reverse=False) [Timsort, O(n log n)], .reverse() [O(n)].
* List Comprehensions: Compact, highly optimized syntax replacing manual loops: [expression for item in iterable if condition].

>> 3.3 Tuples: Immutable Sequences

Tuples are immutable ordered sequences of heterogeneous elements: t = (1, 2, 'three'):
* Tuple Packing and Unpacking: a, b, c = (10, 20, 30); Extended unpacking: first, *middle, last = [1, 2, 3, 4, 5].
* Advantages over Lists: (1) Memory Efficiency: Fixed memory footprint without over-allocation overhead. (2) Immutability: Guarantees data integrity for constant records. (3) Hashability: Tuples can serve as dictionary keys and set elements (if all constituent elements are immutable).

>> 3.4 Dictionaries: Hash Map Architecture and Operations

Dictionaries (dict) are mutable mappings of unique, hashable keys to arbitrary values, implemented as compact open-addressing hash tables with quadratic probing:

* Time Complexity: Average O(1) for lookup, insertion, and deletion.
* Core Methods: .keys(), .values(), .items(), .get(key, default), .pop(key, default), .update(dict2), .setdefault(key, default).
* Dictionary Comprehensions: {key_expr: val_expr for item in iterable if condition}.

>> 3.5 Sets: Hash-Based Unique Collections

Sets (set) are mutable, unordered collections of unique, hashable elements: s = {1, 2, 3}:
* Set Mathematical Operations: Union (|), Intersection (&), Difference (-), Symmetric Difference (^), Subset (<=).

[PROGRAM 1: Text Frequency Analysis & Word Counter]

Problem Context: Build a text-processing utility that cleans raw text (stripping punctuation and converting to lowercase), computes word frequency using a dictionary, and displays the top 5 most frequent words.
Python Source Code:

```
import string

def word_frequency_analyzer(text):
# Clean punctuation and normalize case
clean_text = text.translate(str.maketrans('', '', string.punctuation)).lower()
words = clean_text.split()

freq_map = {}
for word in words:
freq_map[word] = freq_map.get(word, 0) + 1

# Sort by frequency descending using lambda
sorted_words = sorted(freq_map.items(), key=lambda item: item[1], reverse=True)
return sorted_words[:5]

sample = 'Data structures in Python are powerful. Python lists and Python dictionaries offer fast lookups in data science.'
top_words = word_frequency_analyzer(sample)
print('Top 5 Most Frequent Words:')
for word, count in top_words:
print(f'  {word:12s} : {count} occurrences')
```

Execution Trace & Output:
Words: python (3), data (2), in (2), structures (1), are (1).

[PROGRAM 2: Matrix Transposition using List Comprehensions]

Problem Context: Given a 3x4 rectangular matrix, compute its 4x3 transpose using nested list comprehensions.
Python Source Code:

```
matrix = [
[1,  2,  3,  4]
[5,  6,  7,  8]
[9, 10, 11, 12]
]

# Nested list comprehension for transposition
transposed = [[row[col] for row in matrix] for col in range(len(matrix[0]))]

print('Original Matrix (3x4):')
for r in matrix: print(r)
print('\\nTransposed Matrix (4x3):')
for r in transposed: print(r)
```

Execution Trace & Output:
Transposed row 0: [1, 5, 9]; row 1: [2, 6, 10]; row 2: [3, 7, 11]; row 3: [4, 8, 12].

[PROGRAM 3: Student Gradebook with Dictionary Comprehensions]

Problem Context: Implement a gradebook processor calculating percentage marks, assigning letter grades, and filtering distinction holders using dictionary comprehensions.
Python Source Code:

```
scores = {
'Aarav': [88, 92, 79, 95]
'Diya':  [72, 68, 75, 70]
'Rohan': [94, 98, 90, 92]
'Ananya':[55, 62, 58, 60]
}

# Compute average percentage per student
averages = {name: sum(marks) / len(marks) for name, marks in scores.items()}
```

```
# Filter distinction holders (avg >= 85.0)
distinctions = {name: avg for name, avg in averages.items() if avg >= 85.0}

print('All Student Averages:', averages)
print('Distinction Holders (>=85%):', distinctions)
```

Execution Trace & Output:

Averages: Aarav=88.5, Diya=71.25, Rohan=93.5, Ananya=58.75.

Distinction Holders: {'Aarav': 88.5, 'Rohan': 93.5}.

[PROGRAM 4: Set Operations for Common Course Enrollment]

Problem Context: Two university sections have enrolled students in CSE101 and MTH165. Compute: (1) Students taking both courses, (2) Students taking either course, (3) Students in CSE101 only.

Python Source Code:

```
cse_students = {'Aarav', 'Diya', 'Rohan', 'Kavita', 'Suresh'}
math_students = {'Rohan', 'Kavita', 'Neha', 'Priya', 'Amit'}

both_courses = cse_students & math_students          # Intersection
all_students = cse_students | math_students          # Union
cse_only = cse_students - math_students              # Difference
exclusive_to_one = cse_students ^ math_students      # Symmetric Difference

print(f'Enrolled in Both (Intersection):  {both_courses}')
print(f'Total Unique Students (Union):    {all_students}')
print(f'CSE101 Only (Difference):         {cse_only}')
print(f'Taking Exactly One Course:        {exclusive_to_one}')
```

Execution Trace & Output:

Both: {'Rohan', 'Kavita'}; Total unique: 8 students; CSE only: {'Aarav', 'Diya', 'Suresh'}.

>> 3.6 University Review & Examination Questions

* Q1: Contrast lists and tuples in terms of internal memory representation, performance, and mutability.

Answer: Lists are mutable dynamic arrays: elements can be modified, appended, or deleted in-place. Because lists must accommodate growth, CPython over-allocates memory slots, incurring extra pointer overhead and reallocation costs. Tuples are immutable: once

allocated, their length and element references cannot change. Consequently, CPython allocates exact memory blocks for tuples, reducing memory footprint and accelerating creation time. Furthermore, immutable tuples containing only hashable objects are themselves hashable, allowing them to serve as dictionary keys or set elements, whereas lists are unhashable and raise TypeError.

* Q2: Explain the internal hash table architecture of Python dictionaries and how hash collisions are resolved.

Answer: Python dictionaries use a compact hash table structure. Keys are hashed via hash(key) to generate a pseudo-random integer. In modern Python (3.6+), dictionaries maintain two arrays: a sparse indices array and a dense entries array storing [hash, key, value] in insertion order, preserving key insertion order while saving ~30-50% memory. When two distinct keys produce the same hash index (hash collision), Python resolves it using Open Addressing with a pseudo-random perturb sequence: j = ((5*j) + 1 + perturb) % table_size. Lookup continues along the probe chain until the key is found or an empty slot is encountered.

>> 3.7 Key Takeaways and Revision Points

* Strings and tuples are immutable; lists and dictionaries are mutable.
* List append is amortized O(1); insert at index 0 is O(n).
* Dictionary lookups are average O(1) backed by open-addressing hash tables.
* Sets enforce element uniqueness and provide fast O(1) membership testing.